Why a preconditioner M in Algorithm A12 (PCG)?

1 The reason in one line

CG's speed is governed by the spectrum of A. A preconditioner $M \succ 0$, $M \approx A$, replaces that spectrum by the spectrum of $M^{-1}A$, which is clustered near 1. The price is one solve with M per iteration (line 7), so M must be cheap to invert.

2 Why the spectrum matters

Fact (CG error bound, N&W eq. (5.36)). *For $A \succ 0$ with condition number $\kappa(A) = \lambda_{\max}/\lambda_{\min}$, plain CG satisfies*

$$\|x_k - x^*\|_A \;\le\; 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k \|x_0 - x^*\|_A.$$

Moreover, if A has only m distinct eigenvalues, CG terminates in at most m iterations (N&W Thm. 5.4).

Hence, to reduce the A-norm error by a factor ε one needs roughly

$$k \;\approx\; \tfrac{1}{2}\sqrt{\kappa}\,\ln(2/\varepsilon)$$

iterations. A large κ means slow convergence.

3 What M does

Write $M = C^{\mathsf{T}}C$ with C nonsingular and change variables $\hat{x} = Cx$. Then

$$Ax = b \quad\Longleftrightarrow\quad \underbrace{C^{-\mathsf{T}}AC^{-1}}_{\hat{A}}\,\hat{x} = C^{-\mathsf{T}}b.$$

$\hat{A}$ is again symmetric positive definite, so CG applies to it. Algorithm A12 is exactly CG on this transformed system, rewritten in the original variable x, so that C never appears explicitly: only the solve $y = M^{-1}r$ does. Since

$$\hat{A} = C^{-\mathsf{T}}AC^{-1} \;\sim\; C^{-1}C^{-\mathsf{T}}A = M^{-1}A,$$

the convergence of A12 is controlled by $\kappa(M^{-1}A)$ instead of $\kappa(A)$.

Requirements on M.

1. $M \succ 0$: keeps $\hat{A} \succ 0$ and $\delta = r^{\mathsf{T}}M^{-1}r > 0$, so lines 5 and 7 are well defined.
2. $M \approx A$ in the sense that $\kappa(M^{-1}A) \ll \kappa(A)$ (or few eigenvalue clusters).
3. $My = r$ is cheap to solve (diagonal, banded, incomplete Cholesky, ...).

Items 2 and 3 pull in opposite directions: $M = A$ is perfect for 2 but solving with it is the original problem; $M = I$ is perfect for 3 but changes nothing.

4 A hand-sized example

Take $A = \mathrm{diag}(1, 100)$, $b = (1, 100)^\mathsf{T}$, $x^* = (1, 1)^\mathsf{T}$, $x_0 = 0$.

Plain CG ($M = I$). $r_0 = -b = (-1, -100)^\mathsf{T}$, $p_0 = (1, 100)^\mathsf{T}$, $w = Ap_0 = (1, 10^4)^\mathsf{T}$, $\delta = 10001$, $p_0^\mathsf{T} w = 1000001$,

$$\alpha_0 = \frac{10001}{1000001} \approx 0.0100, \qquad x_1 \approx (0.0100,\ 1.0001)^\mathsf{T}.$$

The first component has barely moved: the step is dominated by the large eigenvalue. A second iteration is needed (A has two distinct eigenvalues).

PCG with Jacobi $M = \mathrm{diag}(A)$. $y_0 = M^{-1} r_0 = (-1, -1)^\mathsf{T}$, $p_0 = (1, 1)^\mathsf{T}$, $\delta = r_0^\mathsf{T} y_0 = 101$, $w = Ap_0 = (1, 100)^\mathsf{T}$, $p_0^\mathsf{T} w = 101$,

$$\alpha_0 = 1, \qquad x_1 = (1, 1)^\mathsf{T} = x^*.$$

One iteration: $M^{-1}A = I$ has a single eigenvalue. This example is extreme (here $M = A$), but it isolates the mechanism: *bad scaling* creates spread eigenvalues, and M removes it.

5 A realistic example ($M \neq A$)

Let $n = 100$, $B = \mathrm{tridiag}(-1, 3, -1)$ (eigenvalues in $(1, 5)$, well conditioned), and $D = \mathrm{diag}(d_1, \ldots, d_n)$ with d_i log-spaced from 1 to 1000. Define

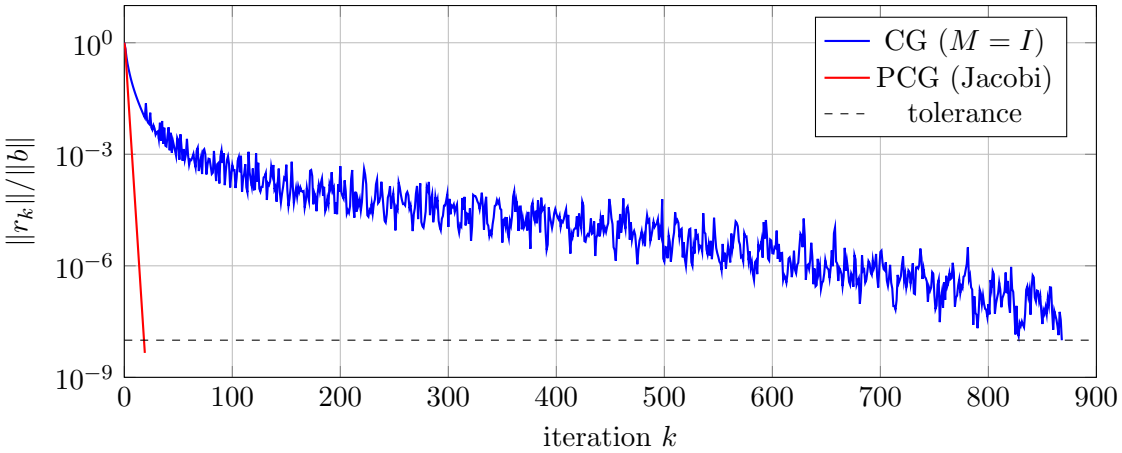
$$A = DBD, \qquad b = A\mathbf{1}, \qquad x^* = \mathbf{1}.$$

A is SPD, but the scaling destroys its conditioning. Take the Jacobi preconditioner $M = \mathrm{diag}(A) = 3D^2$ (solve cost: n divisions). Then

$$M^{-1}A = \tfrac{1}{3}D^{-2}DBD = D^{-1}\big(\tfrac{1}{3}B\big)D \ \sim\ \tfrac{1}{3}B,$$

so the eigenvalues of $M^{-1}A$ lie in $(1/3, 5/3)$ and $\kappa(M^{-1}A) < 5$ *independently of* D.

	κ	iterations to $\|r\| \le 10^{-8}\|b\|$	$\|x_k - x^*\|_2$
CG ($M = I$)	2.5×10^6	868	1.6×10^{-3}
PCG ($M = \mathrm{diag}(A)$)	5.0	19	1.8×10^{-7}



Observations.

- The estimate $k \approx \frac{1}{2}\sqrt{\kappa}\ln(2/\varepsilon)$ with $\varepsilon = 10^{-8}$ gives about 21 for $\kappa = 5$ (observed: 19) and about 1.5×10^4 for $\kappa = 2.5 \times 10^6$.
- In exact arithmetic CG finishes in at most $n = 100$ steps; in floating point, with $\kappa \approx 10^6$, it loses conjugacy and needs 868. Preconditioning also protects against this.
- Each PCG iteration costs one product Ap plus n divisions; total work drops by a factor of about 40.
- Both runs meet the same residual tolerance, yet plain CG's error is 10^4 times larger: with large κ, a small residual does not imply a small error.